Module 02: Laravel Basics & Request Lifecycle

Duration: 4-5 days | **Level:** Beginner

The Request Lifecycle

Every HTTP request in Laravel follows this path:

```
1. public/index.php           -> Entry point
2. bootstrap/app.php          -> Creates Application instance
3. HTTP Kernel                -> Middleware pipeline
4. Router                     -> Matches URI to route
5. Controller / Closure       -> Business logic
6. Response                   -> View, JSON, redirect, file
7. Middleware (outbound)      -> Modify response
8. Send to browser
```

Middleware Pipeline

Middleware wraps your application like layers of an onion:

```
// bootstrap/app.php
->withMiddleware(function (Middleware $middleware): void {
$middleware->alias([
'admin' => \App\Http\Middleware\EnsureUserIsAdmin::class,
]);
})
```

Global middleware runs on every request. **Route middleware** runs only on assigned routes (`auth`, `guest`, `throttle`).

Service Container & Dependency Injection

Laravel's **service container** resolves class dependencies automatically:

```
// Laravel auto-injects DashboardService
public function __construct(private DashboardService $dashboardService) {}
```

Register bindings in `AppServiceProvider`:

```
$this->app->singleton(DashboardService::class, function ($app) {
return new DashboardService();
});
```

Configuration

File	Purpose
config/app.php	App name, timezone, locale
config/database.php	DB connections
config/cache.php	Cache drivers
config/queue.php	Queue drivers
config/mail.php	Mail settings

Access config: `config('app.name')` or `Config::get('app.name')`.

Environment overrides via `.env`: `APP_NAME=TaskForge`.

Facades

Facades provide static syntax for services:

```
Cache::get('key');          // Same as app('cache')->get('key')
Route::get(...);
Auth::user();
```

Service Providers

`bootstrap/providers.php` lists providers. `AppServiceProvider::boot()` is where you register policies, events, and view composers.

TaskForge example - `app/Providers/AppServiceProvider.php`:
- Registers `ProjectPolicy` and `TaskPolicy`
- Maps `TaskCompleted` event to `LogTaskCompletion` listener

Hands-On

1. Run `php artisan route:list` and trace one route through controller -> view.
2. Add a custom middleware that logs every request URL.
3. Use Tinker: `app()->make(DashboardService::class)`.

Next Module

-> Module 03: Routing & Controllers